

Capitulo 07

Optimization

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 10/08/2026

Capítulo 07 — Optimization

Objetivo de aprendizagem: abrir a caixa dos valores que até agora escolhemos "na mão". Vamos entender **inicialização** de pesos, implementar o **AdamW do zero** (e verificar contra o PyTorch), e aplicar **warmup + cosine decay** e **gradient clipping** — medindo quanto se ganha **sem tocar na arquitetura**.

Pré-requisitos: Capítulos 1-6. Especialmente o gradient descent do Cap. 2 e o Transformer do Cap. 5, que é o modelo que vamos treinar melhor aqui.

Arquivos:

- `initialization.py` — de onde vem o $(5/3)/\sqrt{\text{fan_in}}$ do Capítulo 3
- `optimizers.py` — SGD -> momentum -> **AdamW do zero**, verificado
- `train_tuned.py` — baseline vs afinado, comparação controlada
- `exercicios.md` — exercícios

1. O que ficou pendente

Ao longo do curso deixei três dívidas explícitas:

Onde	O que apareceu sem explicação
Cap. 3	<code>W1 = randn(...) * (5/3) / fan_in**0.5</code> — de onde vem esse $5/3$?
Cap. 3	<code>lr = 0.1 if step < 15000 else 0.01</code> — por que decair?
Cap. 5	<code>torch.optim.AdamW(...)</code> — o que ele faz por dentro?

Este capítulo paga as três. E no fim mede o efeito prático: **a mesma arquitetura do Capítulo 5, treinada melhor, chega onde?**

2. Inicialização: o sinal precisa atravessar a rede

Uma rede profunda é uma pilha de multiplicações. Cada camada multiplica a escala do sinal por algum fator. Se esse fator for consistentemente menor que 1, o sinal **encolhe** a cada camada até desaparecer; se for maior que 1, ele **cresce** até saturar ou explodir.

Rodando `python initialization.py`, medimos o desvio padrão das ativações ao longo de 8 camadas (`tanh`, dimensão 256), variando só o **ganho** da inicialização `randn * ganho /  $\sqrt{\text{fan\_in}}$` :

ganho	c1	c2	c4	c6	c8
0,50	0,415	0,201	0,049	0,012	0,003
1,00	0,628	0,486	0,358	0,295	0,254
5/3	0,759	0,694	0,658	0,650	0,653
3,00	0,863	0,840	0,837	0,836	0,838

Leia linha por linha:

- **ganho 0,50** — o sinal morre. Na oitava camada, o desvio padrão é 0,003: as camadas finais recebem praticamente ruído zero, e o gradiente que volta é minúsculo.
- **ganho 1,00** — ainda encolhe (0,628 -> 0,254). Por quê? Porque a **tanh comprime**: $|\tanh(x)| < |x|$ sempre. Ganho 1 não compensa essa perda.
- **ganho 5/3** — estável (0,759 -> 0,653). Este é exatamente o valor que cancela a compressão da **tanh**. **É o 5/3 do Capítulo 3**. Não era mágica: é o *ganho de Kaiming* tabelado para essa ativação.
- **ganho 3,00** — as ativações grudam perto de 0,84, ou seja, a **tanh saturou**. Lembre do Capítulo 2: **tanh** saturada tem derivada ~0. O gradiente morre por um caminho diferente, mas morre.

Por que $\sqrt{\text{fan_in}}$?

Um neurônio soma **fan_in** produtos independentes. A variância de uma soma de **n** termos independentes cresce com **n**, então o **desvio padrão** cresce com $\sqrt{n}$. Dividir por $\sqrt{\text{fan_in}}$ cancela exatamente esse crescimento — e por isso a inicialização funciona igual para camadas de qualquer tamanho.

O ganho depende da ativação

Ativação	ganho	std na camada 1	std na camada 8
tanh	1,000	0,627	0,255
tanh	1,667 (5/3)	0,758	0,653
relu	1,000	0,584	0,056
relu	1,414 ($\sqrt{2}$)	0,825	0,920

A **ReLU** zera metade dos valores, cortando metade da variância; $\sqrt{2}$ recompõe. A regra geral:

```
W ~ randn * ganho / sqrt(fan_in) # inicialização de Kaiming
```

E por que o Capítulo 5 funcionou sem esse cuidado?

Boa pergunta — e a resposta é a **LayerNorm**. Ela re-normaliza as ativações a cada bloco, então o erro de escala não se acumula com a profundidade. Medindo:

ganho	std na camada 8 sem LayerNorm	com LayerNorm
0,50	0,0030	0,4151
1,00	0,2548	0,6271
3,00	0,8381	0,8634

Com LayerNorm, até o ganho catastrófico de 0,50 sobrevive. Isso não dispensa inicializar bem — o começo do treino ainda melhora —, mas explica por que o Transformer do Capítulo 5 treinou sem nenhum ajuste especial. A LayerNorm estava fazendo esse trabalho.

3. Da SGD ao AdamW

O problema: uma learning rate não serve para todos

Imagine dois parâmetros: um cujo gradiente típico é **10**, outro cujo gradiente típico é **0,001**. Com uma learning rate única, você escolhe entre dois males: pequena o bastante para o primeiro não divergir (e então o segundo praticamente não anda), ou grande o bastante para o segundo andar (e então o primeiro explode).

O `optimizers.py` monta esse cenário de propósito (uma loss com termos de escalas 1000x diferentes) e mede quantos passos cada otimizador precisa:

```
SGD : nao chegou em 3000 | termo grande 0.0e+00 | termo pequeno 5.1e-03
SGD + momentum : nao chegou em 3000 | termo grande 0.0e+00 | termo pequeno 5.1e-03
AdamW (nosso) : 711 passos | termo grande 9.4e-04 | termo pequeno 2.6e-14
```

Olhe as duas últimas colunas. As duas versões de SGD resolvem o termo **grande** (chega a zero) e **empacam** no pequeno — por isso param no mesmo lugar. O gargalo não é o momentum: é a learning rate única. O AdamW resolve os dois na mesma corrida.

A escada, degrau por degrau

SGD — anda na direção do gradiente:

```
p -= lr * p.grad
```

Momentum — acumula uma "velocidade". Se o gradiente aponta sempre para o mesmo lado, a velocidade cresce; se ele oscila, as oscilações se cancelam:

```
v = beta * v + (1 - beta) * p.grad
p -= lr * v
```

Adam — a ideia nova: guardar também a média móvel do gradiente **ao quadrado** (v), que estima *quão grande o gradiente daquele parâmetro costuma ser*, e **dividir** o passo por isso. O resultado é um passo cujo tamanho efetivo fica na escala da learning rate, qualquer que seja a escala do gradiente:

```
m = beta1 * m + (1 - beta1) * g # "para onde ir"
v = beta2 * v + (1 - beta2) * g**2 # "quão grande costuma ser"
p -= lr * m_hat / (sqrt(v_hat) + eps)
```

A correção de viés

m e v começam em **zero**. No primeiro passo, $m = (1-\beta_1) \cdot g = 0,1 \cdot g$ — dez vezes menor que o gradiente real. As médias móveis **subestimam** no início. A correção compensa exatamente isso:

```
m_hat = m / (1 - beta1 ** t) # t = número do passo
v_hat = v / (1 - beta2 ** t)
```

No passo 1 com $\beta_1 = 0,9$, o divisor é $1 - 0,9 = 0,1$, o que multiplica m por 10 e recupera a escala. Conforme t cresce, $\beta_1 t \rightarrow 0$ e a correção desaparece sozinha.

O "W": weight decay desacoplado

Weight decay é a regularização L2 que já vimos no Capítulo 1: empurrar os pesos para zero. O Adam original somava esse termo **ao gradiente** — e aí ele passava pela divisão por $\sqrt{v}$, o que distorce a regularização (parâmetros com gradiente grande recebiam menos decay). O **AdamW** aplica o decay **direto no parâmetro**, fora do caminho adaptativo:

```
if self.wd != 0:
    p.mul_(1 - self.lr * self.wd) # decay direto, desacoplado
# ... só depois o passo adaptativo
```

A verificação

Nossa implementação bate com a do PyTorch:

```
=== nosso AdamW vs torch.optim.AdamW (20 passos) ===
passo 1: diferenca maxima = 0.00e+00
passo 5: diferenca maxima = 2.98e-08
passo 20: diferenca maxima = 8.94e-08

resultados batem (atol=1e-6)? True
```

Como no Capítulo 2 (autograd) e no Capítulo 5 (LayerNorm): construímos do zero, e conferimos contra a implementação de referência.

4. Agendamento da learning rate: warmup + cosine

Duas ideias combinadas, ambas com motivo concreto.

Warmup (subir a lr do zero ao pico, nos primeiros passos). O motivo está na seção anterior: no começo, **m** e **v** do Adam quase não têm informação — são estimativas ruins baseadas em pouquíssimos gradientes. Dar passos grandes com base em estimativas ruins é como correr no escuro. O warmup deixa as médias móveis se estabelecerem primeiro.

Cosine decay (descer suavemente até uma fração do pico). No começo do treino, você está longe do mínimo: passos largos são bons. No fim, você está ajustando detalhes: passos largos passam do ponto. Decair resolve isso — é a versão suave e sem "degraus" do `lr = 0.1 if step < 15000 else 0.01` que usei no Capítulo 3.

```
def lr_agendada(step):
    if step < WARMUP_STEPS:
        return base_lr * (step + 1) / WARMUP_STEPS # sobe linearmente
    progresso = (step - WARMUP_STEPS) / (max_steps - WARMUP_STEPS)
    cos = 0.5 * (1 + math.cos(math.pi * progresso)) # vai de 1 a 0
    return base_lr * (MIN_LR_FRAC + (1 - MIN_LR_FRAC) * cos)
```

Um detalhe com consequência: o cosseno precisa saber `max_steps` de antemão. Se você quiser continuar um treino depois de terminado, o agendamento já chegou ao fim e não há como "esticá-lo" sem estragar a curva. É um incômodo real na prática (e assunto do exercício E7).

5. Gradient clipping

De vez em quando um mini-batch produz um gradiente muito maior que o normal — um dado atípico, ou uma região ruim da superfície de erro. Um único passo gigante pode desfazer muito progresso.

O **gradient clipping** limita a **norma** total do gradiente:

```
torch.nn.utils.clip_grad_norm_(modelo.parameters(), GRAD_CLIP)
```

O ponto crucial: ele reescala, mas **não muda a direção**. Se a norma passa do limite, todos os gradientes são multiplicados pelo mesmo fator, de modo que a norma final seja exatamente o limite. A informação de "para onde ir" é preservada; só o tamanho do passo é contido. É uma rede de segurança barata contra picos.

Como escolher o limite (aprendido do jeito difícil)

A primeira versão deste capítulo usava `GRAD_CLIP = 1.0`, e o resultado foi este:

```
clipados: 14841/15000 passos (99%)
```

99% dos passos cortados. Isso não é clipping — é normalização. Com o limite abaixo da norma típica, todo gradiente era reescalado para exatamente 1,0, o que transforma o algoritmo em outra coisa (uma

espécie de *normalized gradient descent*), em vez de proteger contra picos raros.

O erro foi meu, e a lição é prática e generalizável:

Meça antes de escolher. O limite de clipping tem que ficar **acima** da norma típica do gradiente, para capturar apenas as exceções. Rode alguns passos, olhe a distribuição das normas e escolha um limite algumas vezes maior que a mediana.

No nosso caso, a norma média medida é **~1,23** e a máxima ~1,94. Um limite de **1.0** corta tudo; um limite de **3.0** corta só o que realmente foge do padrão. O `train_tuned.py` agora **mede a norma sempre** (mesmo com o clipping desligado) e informa quantos passos foram cortados — exatamente para você poder calibrar.

6. Resultados: uma ablação

O método

Para saber quanto cada técnica contribui, não basta ligar todas e comparar com o baseline — se o resultado mudar, você não sabe a quem atribuir. O procedimento correto é a **ablação**: ligar **uma técnica por vez**, mantendo absolutamente todo o resto igual (mesma arquitetura, mesmos dados, mesma semente, mesmo número de passos). É o mesmo método que usamos no Capítulo 5 para medir o valor das conexões residuais.

As quatro técnicas em teste, mais a combinação de todas:

Técnica	O que muda em relação ao baseline
agendamento	warmup de 500 passos + cosine decay até 10% do pico
clipping	norma do gradiente limitada a 3,0
weight decay	0,1 em vez do default 0,01
init escalada	projeções residuais encolhidas por $1/\sqrt{(2 \cdot n_layer)}$

Os números

Rodando `python train_tuned.py` (~33 minutos na CPU, seis treinos):

Configuração	treino	validação	teste	vs baseline
1. baseline (Cap. 5)	1,7910	1,8114	1,8199	—
2. só agendamento	1,7541	1,7760	1,7816	+0,0355
3. só clipping	1,7910	1,8114	1,8199	0,0000

4. só weight decay 0,1	1,8608	1,8646	1,8727	-0,0532
5. só init escalada	1,7993	1,8187	1,8234	-0,0073
6. tudo junto	1,8063	1,8110	1,8183	+0,0004

Este resultado é mais instrutivo do que um "afinamos e melhorou". Vamos por partes.

O agendamento ganha, e é o único que ganha. **+0,0355** de melhoria — e o valor absoluto de **1,7760** é o **melhor modelo do curso até aqui**, superando o Transformer do Capítulo 5. Isto sem tocar em uma linha da arquitetura. Vale registrar que dos quatro "cuidados de otimização" testados, só este entregou o que prometia.

O clipping é exatamente neutro — e olhe o quanto isso é informativo: as três colunas de loss são **idênticas** às do baseline, até a quarta casa. O motivo está no relatório do próprio script: **clipados: 0/15000 (0%)**. A norma máxima do gradiente em todo o treino foi **1,937**, e o limite era 3,0 — o corte nunca disparou. Ou seja, o clipping aqui é um seguro que nunca foi acionado. Isso não o torna inútil em geral: em modelos e batches maiores, picos de gradiente acontecem, e um único passo descontrolado pode custar horas de treino. Mas neste modelo ele não faz nada, e é honesto dizer isso. (Note também que os números idênticos **validam a medição**: se eles tivessem divergido com zero cortes, haveria um bug em algum lugar.)

O weight decay alto atrapalha — e muito. **-0,0532** é o **maior efeito de toda a tabela**, e é negativo. A explicação está no Capítulo 3: weight decay é uma arma contra *overfitting*, e nós **medimos** que este modelo não está decorando (treino 1,791 vs validação 1,811 — praticamente iguais). Sem *overfitting* para combater, a regularização forte só restringe a capacidade do modelo sem nada em troca. É remédio para uma doença que o paciente não tem.

A init escalada atrapalha um pouco (-0,0073). Ela foi desenhada para modelos **profundos**: com 12 ou mais blocos, as contribuições somadas ao caminho residual se acumulam e precisam ser contidas. Com **3 blocos**, esse acúmulo é pequeno, e encolher as projeções por $1/\sqrt{6} \approx 0,41$ só faz o modelo começar mais fraco sem necessidade.

E "tudo junto" dá quase zero (+0,0004). Agora a razão é transparente: o ganho do agendamento (**+0,0355**) é praticamente anulado pelas perdas do weight decay (**-0,0532**) e da init escalada (**-0,0073**). As contribuições **não se somam** — elas se cancelam.

A lição central deste capítulo: "melhores práticas" não são aditivas nem universais. Cada uma dessas técnicas existe para resolver um problema específico — *overfitting*, instabilidade em redes profundas, picos de gradiente. Aplicá-la quando o problema **não existe** custa desempenho. A única forma de saber é **medir uma por vez**.

E é por isso que a melhor configuração encontrada não é a que tem mais recursos ligados, e sim **baseline + agendamento: 1,7760**.

A inicialização escalada (truque do GPT-2)

Vale entender a técnica que testamos, mesmo tendo saído perdedora aqui. Cada bloco **soma** sua contribuição ao caminho residual (Cap. 5). Com N blocos, essas contribuições se acumulam e o sinal cresce com a profundidade. O GPT-2 compensa encolhendo a inicialização das camadas que *escrevem* no residual, por $1/\sqrt{2 \cdot n_layer}$:

```
escala = (2 * n_layer) ** -0.5
for b in self.blocks:
    for camada in (b.proj, b.ff_out): # as 2 que escrevem no residual
        camada.weight.mul_(escala)
```

São duas escritas por bloco (atenção e feedforward), daí o $2 \cdot n_layer$. Num modelo de 12 blocos o fator é $1/\sqrt{24} \approx 0,20$, e aí ele importa. No nosso, de 3 blocos, atrapalha — o que ilustra exatamente a lição acima.

7. Resumo do capítulo

- **Inicialização:** $\text{randn} * \text{ganho} / \sqrt{\text{fan_in}}$. O $\sqrt{\text{fan_in}}$ neutraliza o tamanho da camada; o **ganho** corrige o efeito da ativação (1 linear, $5/3$ tanh, $\sqrt{2}$ ReLU). Medimos: ganho errado faz o sinal morrer (0,003) ou saturar.
- **LayerNorm** reduz a sensibilidade à inicialização — foi por isso que o Cap. 5 funcionou sem cuidado especial.
- **Adam** guarda duas médias móveis: do gradiente (m , direção) e do seu quadrado (v , escala). Dividir por $\sqrt{v}$ dá um passo adaptado a cada parâmetro — indispensável quando as escalas variam muito.
- **Correção de viés** conserta a subestimação inicial de m e v , e se apaga sozinha.
- **AdamW** aplica weight decay **direto no parâmetro**, fora do caminho adaptativo.
- **Warmup** protege os primeiros passos (quando m e v são estimativas ruins); **cosine decay** troca passos largos por finos conforme o treino avança.
- **Gradient clipping** limita a norma sem mudar a direção — rede de segurança contra picos. **Calibre o limite acima da norma típica**, senão você normaliza todo passo em vez de cortar exceções (foi o erro documentado na Seção 5).
- Nossa implementação de AdamW **bate com a do PyTorch** ($8,9\text{e-}08$).
- **A ablação mudou a conclusão do capítulo.** Das quatro técnicas testadas uma por vez, só o **agendamento** ajudou ($+0,0355$, chegando a **1,7760** — o melhor modelo do curso). O clipping foi neutro (nunca disparou), e o **weight decay alto** ($-0,0532$) e a **init escalada** ($-0,0073$) **pioraram** — cada um por aplicar uma solução a um problema que este modelo não tem.
- **Boas práticas não são aditivas nem universais.** "Tudo junto" deu quase zero porque os ganhos e as perdas se cancelaram. Medir uma por vez é a única forma de saber.

O que vem no Capítulo 8

Fecha aqui a **Fase II**: temos arquitetura, tokenizador e treino afinado. A Fase III ataca outra dimensão: **velocidade**. No **Capítulo 08 — Device** saímos da CPU e levamos o treino para a **GPU**, entendendo

por que a diferença é de ordens de grandeza e o que exatamente muda no código.

-> Antes de seguir, faça os [exercícios](#).

Exercícios — Capítulo 07 (Optimization)

Faça na ordem. Soluções comentadas em [solucoes/](#) — tente antes de olhar.

Aviso de tempo: os exercícios que treinam o Transformer levam ~6 minutos cada na CPU. Reduza `max_steps` (ex.: 4000) para experimentar mais rápido — as *comparações relativas* seguem válidas, só não compare com os números absolutos da apostila.

E1 — Leitura de código (aquecimento)

Sem rodar:

1. No nosso `AdamW`, o que a **correção de viés** (`m / (1 - beta1**t)`) faz? Por que ela importa mais nos primeiros passos e vira irrelevante depois?
2. Qual a diferença entre somar o *weight decay* ao gradiente (Adam original) e aplicá-lo direto no parâmetro (AdamW)? Por que a segunda forma é melhor?
3. Por que `initialization.py` divide por `sqrt(fan_in)` e não por `fan_in`?

E2 — O warmup é necessário?

No `train_tuned.py`, coloque `WARMUP_STEPS = 0` e treine.

1. A loss dos primeiros passos fica pior? O treino chega ao mesmo lugar no fim?
2. Olhe a **norma do gradiente** nos primeiros passos (o script reporta média e máxima). Por que o começo do treino é o momento mais instável?
3. Explique o warmup em uma frase: por que começar com passos pequenos ajuda um otimizador adaptativo como o Adam? (Dica: nos primeiros passos, as médias móveis `m` e `v` têm pouquíssima informação.)

E3 — Calibrando o gradient clipping

O script informa a **norma média** do gradiente e **quantos passos foram clipados**. Use essas duas informações.

1. Com o `GRAD_CLIP = 3.0` do código, que percentual dos passos é cortado? Isso é coerente com "cortar apenas as exceções"?
2. Reduza para `1.0` e depois `0.1`. Em que ponto o clipping deixa de ser uma rede de segurança e passa a ser uma **normalização** de todo passo? (A apostila conta que a primeira versão deste capítulo cortava 99% dos passos — o erro está documentado na Seção 5.)
3. Aumente para `100` (nunca corta). A loss piora? Se não piorar, o que isso diz sobre a necessidade de clipping **neste** modelo?

4. O clipping altera a **direção** do gradiente ou apenas o seu **tamanho**? Por que essa distinção importa?

E4 — O ganho certo para a GELU

O `initialization.py` mostra o ganho correto para `tanh` (5/3) e `ReLU` ($\sqrt{2}$). Nosso Transformer usa **GELU**.

1. Adapte a função `rodar` para aceitar GELU e descubra empiricamente qual ganho mantém o desvio padrão estável ao longo das 8 camadas.
 2. O valor ficou mais perto do da ReLU ou do da tanh? Isso faz sentido, dado que a GELU é uma versão suave da ReLU?
 3. Compare com `torch.nn.init.calculate_gain` — ela tem entrada para GELU? Se não, o que isso sugere na prática?
-

E5 — A curva da learning rate

Treine com `base_lr` = 1e-4, 3e-4, 1e-3, 3e-3 e 1e-2 (reduza `max_steps` para 3000).

1. Monte a tabela de loss de validação. O formato é de um "U"? Onde está o mínimo?
2. O que acontece com `1e-2` — o treino diverge ou só fica ruim?
3. **Você provavelmente vai achar um mínimo em 3e-3, maior que o 1e-3 da apostila.** Isso contradiz o capítulo? (Dica: a apostila treina 15.000 passos, e este exercício 3.000. Com pouco orçamento, compensa andar mais rápido.)

Por que a melhor learning rate depende do **tamanho do batch**? (Pense: com batch maior, o gradiente é menos ruidoso.)

Solução de referência: `solucoes/e5_curva_lr.py`.

E6 — AdamW vs SGD no modelo de verdade

O `optimizers.py` compara os otimizadores num problema de brinquedo. Faça a comparação no Transformer: troque o `torch.optim.AdamW` por `torch.optim.SGD` (com e sem momentum).

1. Com a **mesma** learning rate, o SGD chega perto?
 2. Ajuste a learning rate do SGD para o melhor valor que encontrar. Ele empata com o AdamW?
 3. Por que redes com muitos parâmetros de escalas diferentes (embeddings, pesos de atenção, ganhos de LayerNorm) favorecem otimizadores adaptativos?
-

E7 — Por que o weight decay atrapalhou? (importante)

A ablação da apostila mostra que `weight_decay = 0.1` **piorou** o modelo em `-0,0532` — o maior efeito da tabela, e negativo.

1. Varie o weight decay: `0.0`, `0.01`, `0.1`, `0.5`. Monte a tabela de loss de validação. Existe um valor melhor que o default `0.01`?
2. Compare a loss de **treino** com a de **validação** em cada caso. Quando o weight decay aumenta, as duas sobem juntas ou a de treino sobe mais? O que isso indica?
3. **A pergunta que importa:** weight decay combate *overfitting*. Olhe os números do baseline (treino 1,791 vs validação 1,811). Existe overfitting para combater? Se não existe, o que a regularização forte está fazendo com a capacidade do modelo?
4. Em que cenário você **esperaria** que `weight_decay = 0.1` ajudasse? (Dica: releia a solução do E5 do Capítulo 3, com 155 nomes.)

E8 — Outro agendamento (desafio)

Implemente e compare três agendamentos, todos com o mesmo warmup:

- **constante** (o baseline)
- **cosseno** (o nosso)
- **linear** (decai em linha reta até `MIN_LR_FRAC`)
- **degraus** (divide por 10 em 50% e 80% do treino)

1. Qual vence no nosso problema? A diferença é grande?
2. O cosseno é o padrão em treinos de LLM. Pelos seus números, isso se justifica ou é convenção?
3. Um detalhe importante: o agendamento depende de saber `max_steps` de antemão. Que problema isso cria se você quiser **continuar** um treino já terminado?